



TRƯỜNG ĐẠI HỌC SÀI GÒN

Khoa Công nghệ thông tin

Báo cáo lần 1

DataBridge - Nền tảng kết nối và khai thác cơ sở dữ liệu dựa trên mô hình ngôn ngữ lớn

Môn học: Seminar chuyên đề

GV hướng dẫn: TS. Đỗ Như Tài

Lớp: DCT122C3 – Mã học phần: 841482

Sinh viên: Lê Hồng Phát – 3122411145

Trương Phú Kiệt – 3122411109

Trà Đức Toàn – 3122411217

Lê Ngọc Hiệp – 3122411053

Thành phố Hồ Chí Minh, tháng 5 năm 2026

MỤC LỤC

BẢNG PHÂN CÔNG	2
DANH MỤC CÁC CHỮ VIẾT TẮT	3
DANH MỤC HÌNH ẢNH	Error! Bookmark not defined.
TÓM TẮT ĐỀ TÀI BÁO CÁO	5
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	7
1.1. Tính cấp thiết của đề tài.....	7
1.2. Tổng quan tình hình nghiên cứu.....	7
1.3. Mục tiêu nghiên cứu.....	9
1.4. Nhiệm vụ nghiên cứu.....	10
1.5. Phạm vi và đối tượng nghiên cứu	11
1.6. Phương pháp nghiên cứu	12
1.7. Ý nghĩa khoa học và thực tiễn.....	13
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	15
2.1. Mô hình ngôn ngữ lớn (LLM) và các tương tác	15
2.1.2. Kỹ nghệ câu lệnh (Prompt Engineering) và Chuẩn hóa JSON.....	15
2.2. Bài toán NL2SQL và Multi-Agent	17
2.3. Kiến trúc Microservices và Backend	19
2.4. Hệ quản trị cơ sở dữ liệu và Cơ chế bảo mật Supabase.....	20
2.5. Công nghệ phát triển Giao diện người dùng (Frontend).....	22
2.6. Công nghệ ảo hóa và Quản trị hạ tầng (DevOps).....	25
TÀI LIỆU THAM KHẢO.....	27

BẢNG PHÂN CÔNG

Thành viên	Nội dung công việc	Tiến độ
Lê Hồng Phát	Thiết lập SKILL riêng cho service backend Tổng hợp báo cáo các chương	100%
Trương Phú Kiệt	Thiết lập SKILL cho testing	100%
Trà Đức Toàn	Thiết kế slide Thiết lập SKILL cho frontend, infrastructure	100%
Lê Ngọc Hiệp	Viết báo cáo chương 1, 2 Thiết lập SKILL cho backend	100%

DANH MỤC CÁC CHỮ VIẾT TẮT

STT	Ký hiệu	Ý nghĩa
1	AI	Trí tuệ nhân tạo (Artificial Intelligence)
2	LLM / LLMs	Mô hình ngôn ngữ lớn (Large Language Models)
3	RAG	Sinh văn bản tăng cường truy xuất (Retrieval-Augmented Generation)
4	EdTech	Công nghệ giáo dục (Educational Technology)
5	LMS	Hệ thống quản lý học tập (Learning Management Systems)
6	MCQ	Câu hỏi trắc nghiệm nhiều lựa chọn (Multiple Choice Questions)
7	STT	Nhận diện giọng nói (Speech-to-Text)
8	TTS	Tổng hợp giọng nói / Chuyển đổi văn bản thành giọng nói (Text-to-Speech)
9	gTTS	Thư viện chuyển đổi văn bản thành giọng nói của Google (Google Text-to-Speech)
10	OCR	Nhận dạng ký tự quang học / Đọc văn bản từ ảnh (Optical Character Recognition)
11	API	Giao diện lập trình ứng dụng (Application Programming Interface)
12	CI/CD	Tích hợp và Triển khai liên tục (Continuous Integration / Continuous Delivery hoặc Continuous Deployment)
13	UI/UX	Giao diện người dùng (User Interface) và Trải nghiệm người dùng (User Experience)
14	MVP	Sản phẩm khả dụng tối thiểu (Minimum Viable Product)
15	DB	Cơ sở dữ liệu (Database, ví dụ Vector DB, ChromaDB, MongoDB)
16	RDBMS	Hệ thống quản trị cơ sở dữ liệu quan hệ (Relational Database Management System)
17	GHCR	GitHub Container Registry – Nền tảng lưu trữ container của GitHub

18	KPIs	Chỉ số đo lường hiệu suất (Key Performance Indicators)
19	S3	Chuẩn lưu trữ đối tượng (Object Storage – Simple Storage Service)
20	JSON	Định dạng dữ liệu chuẩn JavaScript Object Notation
21	HTTP	Giao thức truyền tải siêu văn bản (HyperText Transfer Protocol)
22	SSH	Giao thức đăng nhập bảo mật máy chủ (Secure Shell)
23	IP	Địa chỉ giao thức Internet (Internet Protocol)
24	URL	Đường dẫn tài nguyên trên Internet (Uniform Resource Locator)
25	noVNC	Công nghệ truy cập máy tính từ xa qua trình duyệt (Virtual Network Computing via browser)
26	WSL2	Windows Subsystem for Linux phiên bản 2
27	XML	Định dạng dữ liệu mở rộng (eXtensible Markup Language)
28	CSS	Ngôn ngữ định dạng giao diện (Cascading Style Sheets, ví dụ Tailwind CSS)

TÓM TẮT ĐỀ TÀI BÁO CÁO

Hệ thống Agent SQL đã được xây dựng thành công dưới dạng một nền tảng đa tác vụ (Multi-Agent NL2SQL), cho phép chuyển đổi ngôn ngữ tự nhiên thành các truy vấn SQL chính xác và thực thi trực tiếp trên cơ sở dữ liệu Supabase. Về mặt kiến trúc, dự án đã hoàn thiện triển khai mô hình microservices với bốn dịch vụ hoạt động độc lập nhưng phối hợp chặt chẽ thông qua mạng nội bộ Docker. Cụ thể, API Gateway đóng vai trò điều phối trung tâm, NL2SQL Service đảm nhận xử lý ngôn ngữ tự nhiên thông qua các mô hình AI tiên tiến (như GPT-4o Mini và Google Gemini), Query Service phụ trách kết nối cơ sở dữ liệu và Frontend cung cấp giao diện tương tác trực quan cho người dùng. Toàn bộ kiến trúc này đã được quản lý và khởi chạy đồng bộ thông qua Docker Compose, tích hợp tính năng cập nhật mã nguồn theo thời gian thực (Hot Reload) để tối ưu hóa quá trình phát triển.

Thành tựu cốt lõi của hệ thống là luồng xử lý đa tác vụ (Multi-Agent Pipeline) đã được thiết lập vận hành trơn tru với ba tác nhân (Agent) chuyên biệt. Tác nhân Architect đã thực hiện tốt vai trò phân tích ý định người dùng và lập kế hoạch truy vấn dựa trên lược đồ cơ sở dữ liệu (Schema), trong khi tác nhân Generator chịu trách nhiệm sinh mã lệnh PostgreSQL tối ưu kèm theo diễn giải bằng ngôn ngữ tự nhiên. Đặc biệt, hệ thống đã tích hợp thành công tác nhân Validator hoạt động như một chốt chặn cuối cùng, có khả năng tự động kiểm tra tính đúng đắn, sửa các lỗi cú pháp nhỏ và đảm bảo mã SQL tuân thủ nghiêm ngặt các quy định an toàn trước khi chuyển qua bước thực thi.

Đối với khía cạnh bảo mật, hệ thống đã hiện thực hóa một lớp phòng thủ đa tầng (Security Layer) vô cùng chặt chẽ. Cơ chế kiểm duyệt đã được trang bị khả năng sử dụng biểu thức chính quy (Regex) nhằm phát hiện, làm sạch và từ chối các từ khóa nguy hiểm hoặc các đoạn mã mang tính chất tấn công tiêm nhiễm (SQL Injection). Hơn thế nữa, hệ thống quản lý dữ liệu đã thiết lập thành công cơ chế ép buộc mọi kết nối (session) ở chế độ chỉ đọc (read-only), tuyệt đối chặn các lệnh thay đổi cấu trúc bảng, đồng thời áp dụng thành công các giới hạn về thời gian thực thi (timeout tối đa 30 giây) và giới hạn số dòng dữ liệu trả về (mặc định 1000 dòng) để bảo vệ tài nguyên máy chủ.

Về mặt trải nghiệm người dùng, dự án đã hoàn thiện một giao diện tương tác Frontend cao cấp mang đậm phong cách Modern SaaS. Giao diện này được xây dựng bằng framework Next.js, nổi bật với hệ thống nền video lặp động được tối ưu hiệu năng bằng JavaScript thuần (requestAnimationFrame) thay vì CSS tĩnh. Sự kết hợp giữa hiệu ứng

kính mờ (Glassmorphism), hệ thống kiểu chữ (typography) đạt độ chính xác đến từng pixel, cùng với các vùng nhập liệu đa chức năng hỗ trợ đếm tín dụng (credits) và hiển thị công cụ AI đang sử dụng đã mang lại một không gian làm việc chuyên nghiệp, giúp người dùng tra cứu dữ liệu một cách dễ dàng và mượt mà nhất.

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Tính cấp thiết của đề tài.

Trong môi trường kinh doanh hiện đại, việc khai thác thông tin từ các hệ cơ sở dữ liệu (như Supabase Postgres) để đưa ra quyết định nhanh chóng là nhu cầu thiết yếu của các doanh nghiệp. Tuy nhiên, để truy vấn được số liệu, người dùng cần có kỹ năng viết các câu lệnh SQL. Điều này tạo ra rào cản lớn đối với nhóm người dùng nghiệp vụ và các nhà quản lý, khiến họ phải phụ thuộc nhiều vào đội ngũ kỹ sư hoặc nhà phân tích dữ liệu, dẫn đến sự chậm trễ trong quá trình ra quyết định.

Sự ra đời của các Mô hình ngôn ngữ lớn (LLM) đã mang đến giải pháp chuyển đổi ngôn ngữ tự nhiên sang SQL (NL2SQL), giúp người dùng đặt câu hỏi và nhận kết quả một cách trực quan. Mặc dù vậy, việc áp dụng trực tiếp các LLM đơn lẻ vào hệ thống dữ liệu thực tế lại vấp phải những rủi ro lớn. Các mô hình này thường xuyên mắc phải tình trạng "ảo giác AI" (hallucination), sinh ra các truy vấn chứa các bảng hoặc cột không hề tồn tại trong schema dữ liệu. Hơn thế nữa, nếu không có cơ chế kiểm duyệt chặt chẽ, người dùng hoặc AI có thể vô tình hoặc cố ý tạo ra các câu lệnh thay đổi cấu trúc dữ liệu, gây mất an toàn thông tin và ảnh hưởng nghiêm trọng đến sự ổn định của hệ thống.

Từ bối cảnh đó, việc nghiên cứu và xây dựng một nền tảng truy vấn dữ liệu bằng ngôn ngữ tự nhiên đáp ứng được cả hai tiêu chí: thông minh trong xử lý ngôn ngữ và tuyệt đối an toàn trong thực thi, là một yêu cầu vô cùng cấp bách. Đề tài phát triển hệ thống Agent SQL với kiến trúc đa tác vụ (Multi-Agent Pipeline) được ra đời để giải quyết bài toán này. Hệ thống không chỉ tự động đối chiếu schema để ngăn chặn ảo giác AI, mà còn thiết lập một lớp bảo mật đa tầng nghiêm ngặt nhằm cường chế độ chỉ đọc và giới hạn tài nguyên truy vấn. Việc hoàn thành đề tài này sẽ mang lại giá trị thực tiễn to lớn, giúp doanh nghiệp rút ngắn tối đa thời gian tra cứu, loại bỏ rủi ro bảo mật do lỗi viết SQL thủ công và trao quyền tự phục vụ dữ liệu cho các phòng ban không chuyên về kỹ thuật.

1.2. Tổng quan tình hình nghiên cứu

Hiện nay, bài toán chuyển đổi ngôn ngữ tự nhiên sang câu lệnh truy vấn dữ liệu (NL2SQL) đang trải qua một bước tiến mang tính bước ngoặt nhờ sự bùng nổ của các Mô hình ngôn ngữ lớn (LLM) như GPT-4o hay hệ sinh thái Google Gemini. Các mô hình này thể hiện khả năng vượt trội trong việc thấu hiểu ngữ nghĩa, giúp người dùng dễ dàng truy xuất thông tin mà không cần am hiểu về cú pháp SQL.

Tuy nhiên, việc áp dụng trực tiếp các LLM đơn lẻ (Single-Agent) vào hệ thống cơ sở dữ liệu thực tế đang vấp phải những thách thức và rủi ro nghiêm trọng:

- Tình trạng ảo giác AI (Hallucination): Đây là một trong những nhược điểm lớn nhất của các LLM hiện tại. Khi không được định hướng rõ ràng, các mô hình AI thường xuyên tự "tưởng tượng" ra các bảng hoặc cột không hề tồn tại trong schema thực tế, dẫn đến các truy vấn không thể thực thi.
- Lỗ hổng bảo mật và rủi ro hệ thống: Việc để một AI duy nhất vừa tạo mã vừa thực thi tạo ra một điểm mù lớn về bảo mật. Hệ thống có nguy cơ bị đánh lừa bởi các kỹ thuật tiêm nhiễm (SQL Injection) qua các mẫu ký tự đặc biệt (như `;`, `--`, `/*`) hoặc vô tình tạo ra các lệnh làm thay đổi cấu trúc dữ liệu nguy hiểm như `INSERT`, `UPDATE`, `DELETE`, `DROP`.
- Khoảng cách ngữ nghĩa (Semantic Gap): Các thuật ngữ đặc thù của doanh nghiệp thường không được thể hiện rõ trên tên các bảng hay cột. Các hệ thống cũ thường không thể hiểu được các logic nghiệp vụ (business logic) này nếu không được cung cấp thêm ngữ cảnh.

Để giải quyết triệt để những vấn đề trên, xu hướng nghiên cứu trên thế giới đang chuyển dịch mạnh mẽ từ các mô hình đơn lẻ sang kiến trúc Đa tác vụ (Multi-Agent Pipeline). Trong kiến trúc này, nhiệm vụ được chia nhỏ cho các AI chuyên biệt hoạt động độc lập và tự kiểm tra chéo lẫn nhau. Điển hình là quy trình gồm ba bước: một tác nhân chuyên phân tích ý định và trích xuất schema (Architect Agent), một tác nhân chuyên viết mã SQL dựa trên kế hoạch đã lập (Generator Agent), và đặc biệt là một tác nhân kiểm định độc lập (Validator Agent) đóng vai trò chốt chặn để tự động phát hiện, sửa lỗi và loại bỏ mọi truy vấn độc hại trước khi thực thi.

Bên cạnh đó, các nghiên cứu gần đây cũng tập trung tích hợp thêm các công nghệ nhúng (Embeddings) như text-embedding-3-large để xây dựng lớp ngữ nghĩa qua cơ sở dữ liệu Vector (như pgvector), cho phép ứng dụng kỹ thuật RAG để đưa các khái niệm nghiệp vụ vào ngữ cảnh tạo SQL. Đồng thời, các tính năng như Grounding với tìm kiếm thời gian thực cũng bắt đầu được áp dụng để giảm thiểu ảo giác AI.

Dự án Agent SQL được định hình và phát triển dựa trên việc kế thừa những bước tiến công nghệ này. Hệ thống không chỉ ứng dụng kiến trúc Multi-Agent để tối ưu hóa sự chính xác trong bài toán NL2SQL mà còn kết hợp thiết kế Microservices cùng các lớp bảo mật đa tầng (Security Layer) để củng cố chế độ truy vấn chỉ đọc (Read-only) ở

tầng cơ sở dữ liệu. Đây là một hướng tiếp cận toàn diện, vừa khai thác được sức mạnh phân tích của AI, vừa đảm bảo an toàn tuyệt đối cho hệ thống dữ liệu cốt lõi.

1.3. Mục tiêu nghiên cứu

Mục tiêu cốt lõi của đề tài là nghiên cứu và xây dựng thành công Agent SQL - một hệ thống đa tác vụ (Multi-Agent NL2SQL) mạnh mẽ. Hệ thống này hướng tới việc tự động hóa quá trình chuyển đổi các yêu cầu bằng ngôn ngữ tự nhiên thành những câu lệnh truy vấn SQL chính xác, sau đó thực thi chúng một cách an toàn trực tiếp trên hệ quản trị cơ sở dữ liệu Supabase PostgreSQL.

Mục tiêu cụ thể: Để đạt được sự toàn diện cho dự án, đề tài phân rã mục tiêu tổng quát thành các mục tiêu cụ thể trên ba phương diện chính:

- Về mặt kiến trúc và xử lý AI: Xây dựng một hệ thống phần mềm dựa trên kiến trúc Microservices hiện đại (bao gồm API Gateway, NL2SQL Service, Query Service và Frontend) nhằm tối ưu hóa hiệu suất và khả năng mở rộng. Trọng tâm là phải thiết lập được luồng xử lý đa tác nhân (Multi-Agent Pipeline) với sự phối hợp của ba tác nhân: Architect Agent, Generator Agent và Validator Agent để khắc phục tình trạng ảo giác AI và đảm bảo tính chính xác của câu lệnh.
- Về mặt bảo mật và an toàn dữ liệu: Thiết lập một lớp bảo mật đa tầng (Security Layer) vững chắc. Hệ thống phải cường chế quá trình truy vấn hoàn toàn ở chế độ chỉ đọc (Read-only), nghĩa là chỉ cho phép thực thi các lệnh SELECT hoặc WITH CTE, đồng thời từ chối mọi câu lệnh làm thay đổi cấu trúc dữ liệu (như INSERT, UPDATE, DROP). Bên cạnh đó, phải bảo vệ cơ sở dữ liệu khỏi các cuộc tấn công SQL Injection thông qua việc lọc các mẫu ký tự nguy hiểm và kiểm soát chặt chẽ tài nguyên truy vấn (giới hạn thời gian timeout và số lượng dòng trả về).
- Về mặt trải nghiệm người dùng (UI/UX): Đơn giản hóa quá trình phân tích số liệu thông qua một giao diện trực quan mang phong cách Modern SaaS. Mục tiêu là giúp các nhóm người dùng nghiệp vụ (business users) không cần kỹ năng viết SQL vẫn có thể chủ động tra cứu dữ liệu, xem trước các giải thích từ AI và nhận kết quả dưới dạng bảng hoặc biểu đồ. Điều này giúp rút ngắn đáng kể thời gian ra quyết định và trao quyền tự phục vụ dữ liệu cho các phòng ban.

Mục tiêu chiến lược dài hạn: Không chỉ dừng lại ở một công cụ tra cứu tự động, đề tài đặt mục tiêu chiến lược nâng cấp Agent SQL trở thành một Nền tảng Phân tích Dữ liệu Thông minh (Agentic Data Intelligence Platform). Hệ thống hướng tới khả năng không chỉ cung cấp dữ liệu thô mà còn tự động đưa ra các thông tin chuyên sâu (Data Insights), cho phép người dùng đào sâu vấn đề thông qua hội thoại đa bước (Multi-turn Conversation) và tích hợp các lớp ngữ nghĩa (Semantic Layer) để AI thấu hiểu sâu sắc các thuật ngữ kinh doanh đặc thù của doanh nghiệp.

1.4. Nhiệm vụ nghiên cứu

Để hoàn thành các mục tiêu nghiên cứu đã đề ra, đề tài xác định năm nhóm nhiệm vụ trọng tâm cần thực hiện xuyên suốt quá trình phát triển dự án Agent SQL.

Nhiệm vụ đầu tiên là nghiên cứu và thiết kế kiến trúc hệ thống tổng thể theo mô hình Microservices. Cụ thể, hệ thống cần được phân tách thành 4 dịch vụ hoạt động độc lập nhằm tối ưu hóa hiệu suất và khả năng mở rộng: API Gateway làm điểm tiếp nhận và điều phối trung tâm, NL2SQL Service chuyên biệt cho việc xử lý AI, Query Service chịu trách nhiệm kết nối cơ sở dữ liệu, và Frontend cung cấp trải nghiệm tương tác. Toàn bộ hạ tầng này phải được cấu hình, quản lý và đóng gói bằng Docker và Docker Compose, tích hợp cơ chế cập nhật mã nguồn theo thời gian thực (Hot-reload) để đồng bộ hóa môi trường phát triển.

Nhiệm vụ thứ hai tập trung vào cốt lõi trí tuệ nhân tạo: xây dựng luồng xử lý ngôn ngữ tự nhiên đa tác vụ (Multi-Agent Pipeline). Quá trình này đòi hỏi việc thiết kế hệ thống Prompt chặt chẽ và tích hợp API từ các mô hình ngôn ngữ lớn để tạo ra ba tác nhân phối hợp. Architect Agent phải được thiết lập để phân tích ý định người dùng và đối chiếu chính xác với schema cơ sở dữ liệu; Generator Agent chịu trách nhiệm sinh mã lệnh PostgreSQL tối ưu kèm diễn giải; và Validator Agent hoạt động như một màn lọc cuối cùng để tự động kiểm định tính đúng đắn và sửa các lỗi cú pháp nhỏ.

Nhiệm vụ thứ ba là thiết kế và phát triển lớp bảo mật đa tầng (Security Layer) nhằm bảo vệ tuyệt đối cơ sở dữ liệu Supabase. Nhiệm vụ này yêu cầu xây dựng các bộ lọc bằng biểu thức chính quy (Regex) để nhận diện, chặn đứng các từ khóa làm thay đổi cấu trúc dữ liệu (như INSERT, UPDATE, DROP) và các mẫu mã độc có nguy cơ gây SQL Injection. Quan trọng hơn, cần cấu hình hệ thống quản lý kết nối cơ sở dữ liệu (psycopg2) để cưỡng chế mọi session luôn ở chế độ chỉ đọc (readonly=True), đồng thời áp dụng chặt chẽ các giới hạn về thời gian thực thi và số lượng dữ liệu trả về.

Nhiệm vụ thứ tư xoay quanh việc phát triển giao diện người dùng (UI/UX) đạt tiêu chuẩn của các nền tảng Modern SaaS hiện đại. Giao diện frontend cần được xây dựng bằng framework Next.js, kết hợp tinh tế hiệu ứng kính mờ (Glassmorphism) và tuân thủ nghiêm ngặt hệ thống kiểu chữ (typography) đã định nghĩa. Đặc biệt, cần ứng dụng kỹ thuật xử lý JavaScript thuần túy (sử dụng hàm requestAnimationFrame) để quản lý các hiệu ứng chuyển động video nền tĩnh, thay vì phụ thuộc vào CSS thông thường, nhằm mang lại trải nghiệm mượt mà nhất cho người dùng.

Nhiệm vụ cuối cùng là thực hiện kiểm thử và đánh giá toàn diện hệ thống nhằm xác thực tính khả thi và độ ổn định. Đề tài cần triển khai hệ thống kiểm thử đa miền, bao gồm: kiểm thử tính năng giao diện từ đầu đến cuối (E2E Browser Test với Playwright), kiểm thử tích hợp các luồng API (Integration Test), và kiểm thử đo lường hiệu năng chịu tải. Các bước kiểm định này đảm bảo rằng câu lệnh SQL do AI tạo ra không chỉ chính xác với nghiệp vụ mà hệ thống còn vận hành an toàn trong mọi kịch bản sử dụng thực tế.

1.5. Phạm vi và đối tượng nghiên cứu

Hệ thống Agent SQL được thiết kế linh hoạt nhằm đáp ứng nhu cầu khai thác dữ liệu của bốn nhóm đối tượng chính trong doanh nghiệp. Nhóm thứ nhất là những người dùng nghiệp vụ, những người không có nền tảng về ngôn ngữ truy vấn SQL nhưng lại có nhu cầu tra cứu số liệu một cách nhanh chóng bằng ngôn ngữ tự nhiên để đưa ra các quyết định kinh doanh kịp thời. Nhóm thứ hai bao gồm các nhà phân tích dữ liệu mong muốn tối ưu hóa thời gian tạo lập báo cáo từ nhiều bảng liên kết; họ cần một công cụ sinh mã tự động đi kèm khả năng xem trước câu lệnh SQL và các phần giải thích chi tiết để dễ dàng kiểm chứng. Nhóm đối tượng thứ ba là các kỹ sư dữ liệu và quản trị viên hệ thống (DBA), những người đặt sự quan tâm hàng đầu vào tính bảo mật, khả năng kiểm soát tài nguyên và việc ngăn chặn các câu lệnh nguy hiểm. Cuối cùng, hệ thống hướng tới nhóm các trưởng dự án hoặc người quản lý sản phẩm thông qua việc cung cấp công cụ lưu trữ lịch sử truy vấn, biểu đồ hóa dữ liệu và dễ dàng chia sẻ kết quả trong nội bộ nhóm.

Về phạm vi ứng dụng, đề tài tập trung xây dựng một hệ thống kiến trúc Microservices để thực thi trực tiếp các truy vấn trên hệ quản trị cơ sở dữ liệu Supabase PostgreSQL. Chức năng cốt lõi của nền tảng được giới hạn nghiêm ngặt ở các tác vụ truy xuất dữ liệu an toàn, cụ thể là chỉ hỗ trợ AI tạo và thực thi các câu lệnh SELECT hoặc cấu trúc WITH CTE. Nhằm bảo vệ máy chủ, phạm vi nghiên cứu đặc biệt chú trọng vào việc thiết lập

các ranh giới tài nguyên và bảo mật. Hệ thống tự động thiết lập phiên kết nối ở chế độ read-only, chặn các dấu chấm phẩy để ngăn ngừa thực thi đa câu lệnh (multi-statements), đồng thời áp dụng cứng các quy tắc giới hạn truy vấn như thời gian chờ tối đa 30 giây và số lượng kết quả trả về không được vượt quá 1000 dòng.

Bên cạnh đó, nhằm đảm bảo tính toàn vẹn tuyệt đối cho cơ sở dữ liệu gốc, đề tài cũng xác định rõ những khía cạnh nằm ngoài phạm vi thực hiện. Hệ thống hoàn toàn không hỗ trợ việc AI tự động tạo hay thực thi bất kỳ câu lệnh nào làm thay đổi, ghi đè hoặc xóa bỏ dữ liệu cốt lõi. Hơn thế nữa, mặc dù dự án được trang bị lớp kiểm duyệt an toàn, việc quản lý phân quyền truy cập dữ liệu ở mức độ vi mô, như thiết lập Bảo mật cấp độ hàng (Row Level Security - RLS) chi tiết cho từng người dùng, sẽ không do hệ thống AI này tự động can thiệp nếu cơ sở dữ liệu gốc chưa được định cấu hình từ trước.

1.6. Phương pháp nghiên cứu

Để giải quyết trọn vẹn các bài toán đặt ra, đề tài chủ yếu áp dụng phương pháp phân tích và thiết kế hệ thống theo mô hình vi dịch vụ (Microservices) kết hợp cùng kiến trúc Multi-Agent. Thay vì sử dụng một mô hình nguyên khối truyền thống, phương pháp tiếp cận vi dịch vụ cho phép chia tách hệ thống thành bốn phân hệ hoạt động độc lập bao gồm API Gateway, NL2SQL Service, Query Service và Frontend. Cách tiếp cận này giúp cô lập rủi ro bảo mật và tối ưu hóa khả năng mở rộng. Đặc biệt, phân hệ xử lý ngôn ngữ tự nhiên được xây dựng dựa trên phương pháp Multi-Agent Pipeline, phân chia rạch ròi quy trình thành ba bước: phân tích ý định và Architect Agent, Generator Agent và kiểm định, Validator Agent. Sự phối hợp chéo này là phương pháp cốt lõi để loại bỏ tình trạng ảo giác AI (hallucination) thường gặp ở các mô hình đơn lẻ.

Về mặt tích hợp công nghệ và phát triển, đề tài sử dụng phương pháp thực nghiệm kỹ thuật dựa trên các nền tảng trí tuệ nhân tạo tiên tiến. Quá trình sinh mã SQL được thực hiện thông qua việc gọi API từ các Mô hình ngôn ngữ lớn (LLM) như hệ sinh thái Google Gemini và GPT-4o (thông qua OpenRouter). Đề tài ứng dụng chuyên sâu các kỹ thuật Kỹ nghệ Câu lệnh (Prompt Engineering) và định dạng phản hồi chuẩn JSON để ép buộc AI tuân thủ nghiêm ngặt các quy định về cú pháp cơ sở dữ liệu. Song song đó, phương pháp quản lý hạ tầng bằng mã được áp dụng thông qua Docker và Docker Compose. Toàn bộ môi trường phát triển từ Frontend đến Backend đều được container hóa và tích hợp cơ chế nạp lại tức thời (Hot-Reload), đảm bảo tính đồng bộ cao nhất giữa môi trường phát triển cục bộ và môi trường triển khai thực tế.

Đối với khía cạnh đảm bảo an toàn và chất lượng phần mềm, đề tài áp dụng phương pháp tiếp cận bảo mật đa tầng (Multi-Layer Validation) kết hợp cùng chiến lược kiểm thử toàn diện. Phương pháp bảo mật được thiết lập bằng cách sử dụng biểu thức chính quy (Regex) để tự động nhận diện, lọc bỏ các từ khóa hoặc mẫu ký tự nguy hiểm có khả năng gây tấn công tiêm nhiễm (SQL Injection). Chốt chặn an toàn cuối cùng được thực thi bằng phương pháp cấu hình cưỡng chế mọi phiên giao dịch với cơ sở dữ liệu PostgreSQL luôn ở chế độ chỉ đọc (Read-only). Cuối cùng, tính ổn định của nền tảng được kiểm chứng bằng phương pháp kiểm thử đa miền quy mô lớn, bao gồm: kiểm thử chức năng giao diện người dùng đầu cuối, kiểm thử luồng tích hợp (Integration Test thông qua Pytest) và đo lường khả năng chịu tải (Load Test sử dụng k6) nhằm khẳng định độ tin cậy của toàn bộ hệ thống.

1.7. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, đề tài đóng góp một mô hình thiết kế tiên tiến cho bài toán xử lý ngôn ngữ tự nhiên thành câu lệnh truy vấn (NL2SQL) thông qua việc ứng dụng chuyên sâu kiến trúc Đa tác vụ (Multi-Agent Pipeline). Thay vì phụ thuộc vào một mô hình trí tuệ nhân tạo (AI) đơn lẻ vốn dễ mắc phải tình trạng ảo giác (hallucination) và tạo ra các dữ liệu không có thực, nghiên cứu này đã chứng minh tính hiệu quả của việc phân rã quy trình thành ba vai trò độc lập: tác nhân Architect, tác nhân Generator và tác nhân Validator. Hướng tiếp cận này cung cấp một khuôn mẫu tham khảo có giá trị cao trong việc tích hợp an toàn các Mô hình ngôn ngữ lớn (LLM) như Google Gemini hay hệ sinh thái OpenRouter với các hệ quản trị cơ sở dữ liệu quan hệ chặt chẽ, đồng thời khẳng định tính khả thi của việc đóng gói hệ thống AI vào cấu trúc Microservices để tối ưu hóa hiệu suất.

Về mặt thực tiễn đối với hoạt động doanh nghiệp, hệ thống Agent SQL mang lại giá trị to lớn trong việc dân chủ hóa khả năng tiếp cận dữ liệu. Đề tài giúp tháo gỡ rào cản kỹ thuật, trao quyền cho nhóm người dùng nghiệp vụ (business users) – những người không am hiểu cú pháp lập trình – khả năng tự chủ tra cứu, trích xuất và hình ảnh hóa dữ liệu thông qua ngôn ngữ giao tiếp hàng ngày. Giải pháp này giúp rút ngắn tối đa độ trễ từ lúc hình thành câu hỏi nghiệp vụ đến lúc nhận được kết quả phân tích, qua đó đẩy nhanh tiến độ ra quyết định và nâng cao năng lực tự phục vụ (self-service) cho các phòng ban không chuyên.

Bên cạnh đó, đối với đội ngũ kỹ sư và quản trị viên cơ sở dữ liệu (DBA), đề tài mang lại một phương thức vận hành hoàn toàn an tâm. Việc viết các câu lệnh truy vấn thủ công không chỉ tốn thời gian mà còn tiềm ẩn rủi ro phá hoại hệ thống. Hệ thống giải quyết triệt để vấn đề này nhờ áp dụng lớp Multi-Layer Validation, tự động ngăn chặn các nguy cơ tấn công SQL Injection) và cưỡng chế mọi kết nối trực tiếp đến cơ sở dữ liệu Supabase luôn ở trạng thái chỉ đọc (Read-only). Kết quả của đề tài không chỉ tạo ra một công cụ chuyển đổi NL2SQL an toàn, mà còn thiết lập một nền móng hạ tầng vững chắc để doanh nghiệp tiếp tục phát triển hướng tới một Nền tảng Phân tích Dữ liệu Thông minh với các khả năng gợi ý nhận xét hoặc phân tích ngữ nghĩa chuyên sâu trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Mô hình ngôn ngữ lớn (LLM) và các tương tác

Trong kỷ nguyên của Trí tuệ nhân tạo sinh tạo (Generative AI), các Mô hình ngôn ngữ lớn (LLM) đóng vai trò là hạt nhân cốt lõi trong việc thấu hiểu và xử lý ngôn ngữ tự nhiên.

Việc áp dụng các mô hình này vào bài toán truy vấn cơ sở dữ liệu đòi hỏi hệ thống phải tích hợp nhiều kỹ thuật tương tác đặc thù. Điều này nhằm tối ưu hóa độ chính xác, tính an toàn và khả năng tự động hóa luồng công việc.

2.1.1. Sự tích hợp đa dạng các Mô hình ngôn ngữ lớn

Hệ thống Agent SQL được thiết kế với kiến trúc vi dịch vụ linh hoạt để không phụ thuộc vào một mô hình AI đơn lẻ, mà tận dụng sức mạnh từ nhiều nền tảng tiên tiến.

Điển hình, dự án tích hợp sâu rộng hệ sinh thái Google Gemini thông qua bộ công cụ phát triển phần mềm (SDK) google-genai mới nhất. Hệ thống hỗ trợ đa dạng các phiên bản từ Gemini 2.5 Pro, Gemini 2.5 Flash, cho đến các bản thử nghiệm tối tân như Gemini 3.1 Pro Preview và Gemini 3.1 Flash Preview.

Song song với đó, dự án ứng dụng nền tảng OpenRouter đóng vai trò như một cổng giao tiếp thống nhất, tương thích hoàn toàn với tiêu chuẩn OpenAI SDK. Thông qua cổng kết nối này, hệ thống có thể điều phối và khai thác các mô hình ngôn ngữ hàng đầu khác trên thị trường, chẳng hạn như GPT-4o Mini để xử lý cả văn bản và hình ảnh đa phương thức (Multimodal).

Bên cạnh đó, hệ thống còn mở rộng khả năng xử lý bằng cách tích hợp Groq API để ứng dụng mô hình mã nguồn mở Whisper trong việc chuyển đổi âm thanh sang văn bản (Speech-to-Text) với tốc độ cao. Đồng thời, hệ thống cũng sử dụng Tavily API làm tác nhân tìm kiếm web chuyên dụng.

2.1.2. Kỹ nghệ câu lệnh (Prompt Engineering) và Chuẩn hóa JSON

Để các mô hình ngôn ngữ lớn có thể vận hành trơn tru trong một hệ thống tự động không có sự can thiệp của con người, Prompt Engineering là yếu tố mang tính quyết định.

Do bản chất AI thường có xu hướng phản hồi bằng các đoạn hội thoại dài dòng hoặc định dạng văn bản markdown, hệ thống phải áp dụng các quy tắc kiểm soát đầu ra cực kỳ nghiêm ngặt. Trọng tâm của kỹ thuật này là ép buộc AI chỉ được phép trả về định dạng chuẩn JSON thuần túy.

Cụ thể, trong hệ thống Đa tác vụ (Multi-Agent Pipeline), tác nhân phân tích ý định (Architect Agent) được yêu cầu phản hồi một cấu trúc JSON chứa danh sách các bảng dữ liệu cần thiết và một kế hoạch truy vấn chi tiết.

Kế tiếp, Generator Agent sẽ xuất ra mã SQL PostgreSQL kèm theo phần giải thích bằng ngôn ngữ tự nhiên. Trong khi đó, Validator Agent có nhiệm vụ trả về các thông số đánh giá tính đúng đắn và an toàn cùng với đoạn mã SQL đã được sửa lỗi.

Hơn nữa, system prompts cũng chứa các quy tắc bảo mật cứng. Hệ thống tuyệt đối cấm AI sử dụng dấu chấm phẩy để ngăn ngừa lỗi đa câu lệnh, đồng thời chặn mọi lệnh thao tác dữ liệu như INSERT, UPDATE hay DROP.

2.1.3. Kỹ thuật Grounding và Cơ chế giảm thiểu Ảo giác AI (Hallucination)

Một trong những thách thức lớn nhất khi áp dụng trí tuệ nhân tạo là tình trạng ảo giác AI (hallucination). Đây là hiện tượng mô hình tự suy diễn thông tin hoặc tự tạo ra các bảng, cột dữ liệu không hề tồn tại trong hệ thống.

Để khắc phục hiện tượng này đối với các thông tin bên ngoài, dự án tích hợp tính năng tìm kiếm theo thời gian thực (Google Search Grounding). Tính năng này cho phép các mô hình Gemini kết nối trực tiếp với công cụ tìm kiếm trên Internet để truy xuất thông tin mới nhất và đối chiếu thực tế.

Kết quả trả về từ mô hình sẽ được đính kèm các metadata chứa liên kết nguồn trích dẫn. Điều này cho phép hệ thống tự động chèn các liên kết này trực tiếp vào câu trả lời để xác thực tính minh bạch của thông tin.

Đối với phạm vi cơ sở dữ liệu nội bộ, cơ chế chống ảo giác được thực thi nghiêm ngặt thông qua tác nhân kiểm định. Hệ thống yêu cầu AI phải đối chiếu mã SQL với các bảng thực tế. AI chỉ được phép sử dụng các bảng, cột đã được định nghĩa trong schema.

2.1.4. Lớp ngữ nghĩa (Semantic Layer) và Tìm kiếm tăng cường ngữ cảnh (RAG)

Nhằm giúp AI thấu hiểu sâu sắc các logic nghiệp vụ đặc thù của doanh nghiệp thay vì chỉ nhận diện dữ liệu ở mức độ bề mặt, hệ thống áp dụng kỹ thuật tạo lập vector nhúng (Embeddings) kết hợp cùng phương pháp Tìm kiếm tăng cường ngữ cảnh (Retrieval-Augmented Generation - RAG).

Quá trình này sử dụng các mô hình nhúng chuyên dụng như text-embedding-3-large hoặc text-embedding-3-small thông qua cổng OpenRouter. Nhờ đó, các văn bản sẽ được chuyển hóa thành các vector số học.

Các vector này sau đó được lưu trữ trong cơ sở dữ liệu hỗ trợ tìm kiếm không gian. Mục tiêu là nhằm xây dựng một Từ điển thuật ngữ (Glossary) hay Lớp ngữ nghĩa (Semantic Layer).

Nhờ đó, trước khi hệ thống sinh ra câu lệnh SQL, AI có thể tự động tra cứu định nghĩa về các khái niệm kinh doanh phức tạp. Việc đưa những logic này vào quá trình truy vấn đảm bảo kết quả trả về hoàn toàn khớp với thực tiễn hoạt động của doanh nghiệp.

2.2. Bài toán NL2SQL và Multi-Agent

Bài toán chuyển đổi ngôn ngữ tự nhiên sang câu lệnh truy vấn (NL2SQL) là thách thức cốt lõi của hệ thống, nhằm giúp những người dùng nghiệp vụ không có nền tảng kỹ thuật vẫn có thể khai thác cơ sở dữ liệu một cách dễ dàng,.

Tuy nhiên, nếu chỉ sử dụng một mô hình AI duy nhất (Single-Agent) để vừa đọc hiểu yêu cầu, vừa viết mã và trực tiếp gửi đi thực thi, hệ thống sẽ phải đối mặt với rủi ro bảo mật nghiêm trọng và tình trạng "ảo giác AI" tạo ra các truy vấn không có thực,.

2.2.1. Tổng quan Kiến trúc Đa tác vụ (Multi-Agent Pipeline)

Để giải quyết triệt để những hạn chế của phương pháp truyền thống, dự án Agent SQL đã ứng dụng kiến trúc Đa tác vụ tại dịch vụ xử lý ngôn ngữ tự nhiên.

Kiến trúc này chia tách nhiệm vụ xử lý thành ba giai đoạn độc lập nhằm tối ưu hóa độ chính xác và đảm bảo an toàn. Hệ thống được thiết kế để duy trì luồng dữ liệu một cách nghiêm ngặt, tuyệt đối không cho phép bỏ qua bất kỳ bước nào trong chuỗi ba tác nhân: Architect, Generator và Validator.

2.2.2. Tác nhân Phân tích Ý định (Architect Agent)

Đứng ở vị trí đầu tiên trong chuỗi xử lý là Architect Agent. Tác nhân này chịu trách nhiệm tiếp nhận và phân tích sâu sắc câu hỏi bằng ngôn ngữ tự nhiên của người dùng để xác định chính xác ý định (intent) cốt lõi.

Thay vì vội vàng sinh mã ngay lập tức, Architect Agent sẽ đối chiếu yêu cầu với Lược đồ cơ sở dữ liệu (Schema) thực tế. Từ đó, nó chọn lọc và trích xuất ra một danh sách các bảng dữ liệu thực sự cần thiết cho tác vụ.

Sau khi xác định được các bảng nguồn, tác nhân này tiến hành xây dựng một Kế hoạch truy vấn (query_plan) chi tiết. Kế hoạch này phác thảo trước các logic kết nối bảng (Join) và các hàm tổng hợp (Aggregation), sau đó được đóng gói dưới định dạng JSON thuần túy để chuyển giao cho bước tiếp theo,.

2.2.3. Tác nhân Sinh mã lệnh (SQL Generator Agent)

Tiếp nối quy trình, tác nhân Generator sẽ đóng vai trò như một lập trình viên cơ sở dữ liệu. Dựa trên bản kế hoạch truy vấn chi tiết và danh sách các bảng đã được cung cấp từ Architect Agent, tác nhân này tiến hành khởi tạo câu lệnh PostgreSQL thực tế.

Để đảm bảo hiệu suất thực thi và tính dễ đọc của mã, Generator Agent được hướng dẫn sử dụng các bí danh bảng (Alias như u, o, p) giúp câu lệnh SQL trở nên ngắn gọn và tối ưu nhất. Bên cạnh việc sinh mã, một nhiệm vụ rất quan trọng khác của tác nhân này là tạo ra các đoạn giải thích (explanation) bằng ngôn ngữ tự nhiên. Điều này giúp người dùng cuối (đặc biệt là các nhà phân tích dữ liệu) dễ dàng đọc hiểu cấu trúc mã và an tâm hơn trước khi thực thi truy vấn.

2.2.4. Tác nhân Kiểm định và Sửa lỗi (Validator Agent)

Chốt chặn cuối cùng và cũng là tác nhân quan trọng nhất trong việc bảo vệ cơ sở dữ liệu là Validator Agent. Đây là lớp đánh giá độc lập nhằm kiểm tra tính đúng đắn (correctness) của câu lệnh được tạo ra so với yêu cầu ban đầu của người dùng.

Về mặt an toàn, Validator Agent thực thi các quy tắc chống "ảo giác AI" một cách khắt khe. Nó phải đối chiếu và xác minh rằng đoạn mã SQL được sinh ra chỉ sử dụng các bảng và cột thực sự tồn tại trong lược đồ dữ liệu đã cung cấp.

Tác nhân này cũng được huấn luyện để ngăn chặn tiêm nhiễm (anti-injection), tự động phát hiện các yêu cầu của người dùng cố tình lách luật hệ thống. Trong trường hợp phát hiện các sai sót nhỏ về cú pháp, tác nhân này có khả năng tự động sửa lỗi và trả về đoạn mã đã hoàn thiện.

2.2.5. Các quy tắc an toàn cốt lõi trong luồng Đa tác vụ

Bên cạnh việc phân chia rõ ràng trách nhiệm của từng tác nhân, toàn bộ đường ống xử lý phải tuân thủ nghiêm ngặt một bộ quy tắc bảo mật chung, được truyền vào hệ thống thông qua các system prompts.

Quy tắc tối thượng là hợp đồng "Chỉ đọc" (SELECT-Only). Hệ thống tuyệt đối từ chối việc nới lỏng quy định này và cấm hoàn toàn các câu lệnh làm thay đổi cấu trúc dữ liệu như INSERT, UPDATE, DELETE, hay DROP,.

Ngoài ra, hệ thống tự động ép buộc kiểm soát tài nguyên. Nếu người dùng không chỉ định số lượng dòng muốn xem, AI được yêu cầu phải tự động gắn thêm mệnh đề LIMIT 100 để tránh tình trạng tràn dữ liệu.

Cuối cùng, AI tuyệt đối không được sử dụng dấu chấm phẩy (;) để loại bỏ nguy cơ chèn đa câu lệnh (Multiple Statements). Đồng thời, chất lượng kết nối dữ liệu được kiểm soát chặt chẽ bằng cách bắt buộc sử dụng INNER JOIN hoặc LEFT JOIN với các mệnh đề ON rõ ràng, tránh hiện tượng truy vấn chéo (Cross Join) gây sập hệ thống.

2.3. Kiến trúc Microservices và Backend

Thay vì sử dụng một kiến trúc monolithic truyền thống, dự án Agent SQL được thiết kế hoàn toàn dựa trên mô hình Microservices hiện đại. Mục tiêu cốt lõi của kiến trúc này là phân tách ranh giới bảo mật rõ ràng giữa việc xử lý ngôn ngữ bằng trí tuệ nhân tạo và việc thực thi câu lệnh trực tiếp trên cơ sở dữ liệu.

Hệ thống được chia thành bốn phân hệ dịch vụ hoạt động hoàn toàn độc lập, giao tiếp với nhau thông qua mạng nội bộ HTTP của môi trường Docker, đảm bảo không gọi trực tiếp ra bên ngoài khi không cần thiết.

2.3.1. Các phân hệ vi dịch vụ lõi

Phân hệ đầu tiên là API Gateway hoạt động cố định tại cổng mạng 8000. Đây là điểm tiếp nhận trung tâm của toàn bộ hệ thống, chịu trách nhiệm điều phối luồng dữ liệu giữa giao diện người dùng và các dịch vụ xử lý phía sau, đồng thời đảm nhận nhiệm vụ quản lý lịch sử các lượt truy vấn.

Phân hệ thứ hai là Dịch vụ xử lý ngôn ngữ tự nhiên (NL2SQL Service) vận hành tại cổng 8002. Đây là nơi chứa bộ não trí tuệ nhân tạo với luồng Đa tác vụ (Multi-Agent Pipeline) sử dụng các mô hình ngôn ngữ lớn như OpenRouter hoặc Google Gemini. Nhiệm vụ duy nhất của dịch vụ này là phân tích ý định, kiểm tra lược đồ cơ sở dữ liệu và tạo ra câu lệnh SQL tối ưu.

Phân hệ thứ ba là Dịch vụ Truy vấn (Query Service) chạy ở cổng 8001. Dịch vụ này đóng vai trò như một người gác cổng, đảm nhận việc thiết lập kết nối bảo mật tới hệ quản trị Supabase PostgreSQL và thực thi các câu lệnh SQL đã được kiểm duyệt. Hệ thống áp dụng quy tắc cứng để tuyệt đối duy trì sự tách biệt, không bao giờ hợp nhất trách nhiệm giữa dịch vụ NL2SQL và dịch vụ Query.

Cuối cùng là phân hệ giao diện người dùng (Frontend) hoạt động tại cổng 3000. Phân hệ này được xây dựng bằng công nghệ Next.js, cung cấp trải nghiệm bảng điều khiển (Dashboard) trực quan và mượt mà cho người sử dụng.

2.3.2. Công nghệ phát triển API (FastAPI & Pydantic)

Về nền tảng công nghệ Backend, toàn bộ các dịch vụ hệ thống được phát triển chủ yếu bằng ngôn ngữ lập trình Python.

Dự án ứng dụng FastAPI làm framework chính để xây dựng hệ thống nhờ vào tốc độ xử lý API bất đồng bộ vượt trội. FastAPI được kết hợp chặt chẽ cùng thư viện Pydantic, đóng vai trò tự động xác thực và chuẩn hóa cấu trúc dữ liệu đầu vào một cách an toàn.

2.3.3. Quản lý kết nối cơ sở dữ liệu với Psycopg2

Để giao tiếp với cơ sở dữ liệu hệ quản trị Supabase (PostgreSQL), hệ thống ứng dụng kỹ thuật Connection Pooling (sử dụng Pooler Host ở chế độ Session Mode) để hỗ trợ ổn định giao thức kết nối IPv4 bên trong môi trường Docker.

Tại tầng Dịch vụ Truy vấn, thư viện Psycopg2 được tin dùng làm trình điều khiển cốt lõi. Quá trình duy trì kết nối được quản lý thông qua cơ chế contextmanager của Python. Điều này đảm bảo rằng mọi phiên giao dịch (connection) luôn được đóng lại một cách an toàn kể cả khi hệ thống gặp sự cố đột xuất.

Nhằm tối ưu hóa luồng dữ liệu trả về Frontend, hệ thống sử dụng công cụ RealDictCursor của thư viện Psycopg2. Công cụ này giúp định dạng kết quả truy vấn gốc thành danh sách các từ điển thay vì các mảng dữ liệu thô nguyên thủy.

Hơn thế nữa, một lớp chuyển đổi dữ liệu an toàn (Type-Safe Serialization) cũng được thiết lập. Nhờ lớp chuyển đổi này, các kiểu dữ liệu đặc thù của cơ sở dữ liệu PostgreSQL như số thập phân (Decimal) hay định dạng thời gian (DateTime) đều được tự động chuyển đổi thành dạng chuỗi văn bản hoặc chuẩn ngày tháng ISO. Kỹ thuật này giúp dữ liệu dễ dàng được đóng gói dưới định dạng JSON-serializable để truyền mượt mà lên giao diện Frontend mà không gây ra lỗi.

2.4. Hệ quản trị cơ sở dữ liệu và Cơ chế bảo mật Supabase

Trong hệ thống Agent SQL, việc quản lý và bảo vệ dữ liệu đóng vai trò sống còn. Đề tài lựa chọn hệ quản trị cơ sở dữ liệu Supabase làm nền tảng lưu trữ cốt lõi, kết hợp cùng một loạt các cơ chế bảo mật nghiêm ngặt nhằm đảm bảo an toàn tuyệt đối trước mọi rủi ro tấn công.

2.4.1. Nền tảng Supabase PostgreSQL và Cơ chế Connection Pooling

Dự án sử dụng nền tảng Supabase, một hệ sinh thái mã nguồn mở mạnh mẽ được xây dựng dựa trên lõi cơ sở dữ liệu PostgreSQL,. Nền tảng này không chỉ cung cấp khả năng

lưu trữ dữ liệu quan hệ truyền thống mà còn hỗ trợ các hệ thống tiên tiến, tạo tiền đề vững chắc cho việc thực thi các câu lệnh SQL phức tạp.

Để giải quyết các rào cản về giao thức mạng khi triển khai trên môi trường ảo hóa Docker, hệ thống ứng dụng kỹ thuật quản lý nhóm kết nối (Connection Pooling). Cụ thể, dự án sử dụng Pooler Host ở chế độ phiên (Session Mode), giúp duy trì tính ổn định của các giao thức kết nối IPv4 giữa các dịch vụ vi mô và máy chủ cơ sở dữ liệu,.

2.4.2. Lớp bảo mật đa tầng và Chống tiêm nhiễm mã độc (SQL Injection)

Nhằm ngăn chặn triệt để các hành vi phá hoại hoặc thao tác sai lệch, hệ thống thiết lập một Lớp bảo mật đa tầng (Multi-Layer Validation) hoạt động như một màng lọc chặt chẽ trước khi bất kỳ truy vấn nào chạm đến cơ sở dữ liệu.

Cơ chế này sử dụng biểu thức chính quy (Regex) để thực hiện kiểm tra từ khóa (Keyword Check). Hệ thống tự động từ chối và chặn đứng mọi lệnh có khả năng làm thay đổi cấu trúc dữ liệu như các lệnh thao tác thêm, sửa, xóa, hoặc thay đổi bảng (INSERT, UPDATE, DROP, ALTER).

Bên cạnh đó, cơ chế kiểm tra mẫu mã độc (Pattern Check) được triển khai để nhận diện và loại bỏ các kỹ thuật tấn công tiêm nhiễm (SQL Injection). Hệ thống nghiêm cấm việc sử dụng dấu chấm phẩy (;) nhằm ngăn chặn nguy cơ tin tặc chèn và thực thi nhiều câu lệnh cùng lúc (multiple statements). Đồng thời, các ký hiệu chú thích trong SQL (như -- hoặc /*) cũng bị vô hiệu hóa để chặn đứng các kỹ thuật đánh lừa hệ thống.

Để tăng độ chính xác của bộ lọc, hệ thống còn tích hợp chức năng bảo vệ chuỗi ký tự (String Literal Protection). Cơ chế này giúp bóc tách và phân biệt rõ ràng đâu là câu lệnh, đâu là nội dung văn bản. Nhờ đó, các từ khóa nằm bên trong chuỗi văn bản hợp lệ của người dùng sẽ không bị hệ thống nhận diện nhầm là câu lệnh độc hại.

2.4.3. Cơ chế cưỡng chế Chỉ đọc (Read-Only) và Giới hạn tài nguyên

Chốt chặn an toàn cuối cùng và quan trọng nhất của hệ thống nằm ở cơ chế quản lý phiên kết nối của Dịch vụ Truy vấn. Thông qua trình điều khiển thư viện kết nối, hệ thống ép buộc mọi phiên làm việc với cơ sở dữ liệu phải luôn hoạt động ở trạng thái chỉ đọc (readonly=True),. Ngay cả khi một câu lệnh chứa nguy cơ lọt qua được lớp kiểm duyệt văn bản, nó vẫn sẽ bị máy chủ cơ sở dữ liệu từ chối nếu mang ý đồ thay đổi thông tin.

Song song với việc bảo vệ dữ liệu, hệ thống cũng áp dụng các biện pháp bảo vệ tài nguyên máy chủ. Mọi truy vấn đều phải tuân thủ nghiêm ngặt giới hạn về thời gian thực thi (timeout) với ngưỡng tối đa là 30 giây,.

Đồng thời, hệ thống cũng giới hạn số lượng kết quả trả về không được vượt quá một ngưỡng nhất định (mặc định tối đa 1000 dòng). Nếu người dùng đưa ra một truy vấn quá lớn, quy tắc này sẽ tự động ép AI hoặc hệ thống bổ sung các mệnh đề giới hạn để ngăn chặn tình trạng tràn bộ nhớ.

2.4.4. Bảo mật cấp độ hàng (RLS) và Quản trị quyền truy cập

Bên cạnh các cơ chế phòng thủ trực tiếp từ ứng dụng, hệ thống tận dụng triệt để kiến trúc bảo mật dữ liệu ở tầng cơ sở thông qua tính năng RLS (Row Level Security) của PostgreSQL,.

Khi tính năng RLS được kích hoạt, nó đảm bảo rằng người dùng chỉ có thể xem được những dòng dữ liệu mà hệ thống phân quyền cho phép. Điều này có nghĩa là trong trường hợp AI tạo ra một câu lệnh truy vấn có phạm vi quét toàn bộ bảng dữ liệu, cơ chế RLS sẽ đóng vai trò như một bộ lọc ngầm, tự động cắt xén và chỉ trả về các thông tin hợp lệ với phiên đăng nhập của người dùng đó.

Hơn thế nữa, các nhà phát triển cũng phải tuân thủ các quy tắc bảo mật khắt khe khi thiết kế cơ sở dữ liệu. Điển hình là việc áp dụng các chính sách ủy quyền (security invoker) đối với các khung nhìn (views) để ngăn chặn hiện tượng khung nhìn tự động vượt rào RLS, qua đó triệt tiêu mọi lỗ hổng bảo mật tiềm ẩn trong quá trình truy xuất thông tin.

2.5. Công nghệ phát triển Giao diện người dùng (Frontend)

Giao diện người dùng đóng vai trò là điểm chạm trực tiếp giữa hệ thống dữ liệu phức tạp và người sử dụng. Trong dự án Agent SQL, phân hệ này được thiết kế và phát triển dựa trên các tiêu chuẩn khắt khe nhất của các nền tảng dịch vụ phần mềm hiện đại nhằm mang lại trải nghiệm tra cứu dữ liệu mượt mà, trực quan và tối ưu hóa hiệu suất.

2.5.1. Nền tảng Công nghệ cốt lõi (Core Tech Stack)

Toàn bộ kiến trúc giao diện của dự án được xây dựng trên nền tảng framework Next.js kết hợp cùng thư viện React. Việc ứng dụng mô hình định tuyến App Router của Next.js giúp hệ thống tối ưu hóa tốc độ tải trang và hỗ trợ linh hoạt các phương thức kết xuất giao diện.

Đề định hình phong cách trực quan một cách nhanh chóng và đồng bộ, dự án sử dụng framework TailwindCSS. Nền tảng này cho phép các nhà phát triển can thiệp sâu vào từng chi tiết hiển thị trên giao diện mà không cần phải viết các tệp CSS tùy chỉnh đồ sộ, đồng thời hỗ trợ thiết lập một hệ thống biến số thống nhất cho toàn bộ dự án. Toàn bộ phân hệ Frontend được cấu hình để hoạt động độc lập tại cổng mạng 3000, giao tiếp mượt mà với các dịch vụ Backend thông qua kiến trúc Microservices.

2.5.2. Phong cách Thiết kế Modern SaaS và Hiệu ứng Glassmorphism

Định hướng thẩm mỹ của hệ thống tuân thủ nghiêm ngặt tiêu chuẩn giao diện Modern SaaS, ưu tiên giao diện nền sáng kết hợp cùng các hiệu ứng nổi bật tinh tế.

Đặc trưng cốt lõi của phong cách này là việc áp dụng triệt để hiệu ứng kính mờ (Glassmorphism). Cụ thể, các bảng điều khiển hoặc vùng nhập liệu (như Search Input Box) được thiết lập thuộc tính làm mờ nền (backdrop-blur) kết hợp với lớp phủ tối trong suốt, ví dụ như mã màu `rgba(0,0,0,0.24)`. Kỹ thuật này giúp các khối chức năng tách biệt rõ ràng khỏi không gian nền mà vẫn duy trì được chiều sâu thị giác. Hơn thế nữa, để đảm bảo tính đồng nhất trên mọi nền tảng trình duyệt, hệ thống luôn thiết lập các tiền tố đặc thù nhằm khắc phục triệt để các lỗi mất hiệu ứng hiển thị trên trình duyệt Safari.

Bên cạnh đó, hệ thống phân cấp các nút bấm cũng được quy định rõ ràng. Nút hành động chính luôn sử dụng nền đen, chữ trắng; các nút phụ sử dụng nền trong suốt hoặc xám nhạt; và các nút mang tính chất kêu gọi nổi bật được áp dụng các dải màu xanh lá cây tương phản.

2.5.3. Hệ thống Kiểu chữ (Typography) và Bố cục không gian (Pixel-Perfect Spacing)

Dự án không sử dụng các phong chữ mặc định mà thiết lập một hệ thống Typography chuyên biệt để tạo nên bản sắc riêng. Bốn bộ phong chữ cốt lõi được hệ thống nhập liệu bao gồm Schibsted Grotesk, Inter, Noto Sans và Fustat, hỗ trợ đa dạng các độ dày từ 400 đến 700.

Việc xử lý kiểu chữ yêu cầu độ chính xác cao đến từng điểm ảnh (pixel-perfect). Chẳng hạn, các tiêu đề chính (Headline) bắt buộc phải sử dụng phong Fustat in đậm với kích thước 80px, đi kèm với cấu hình khoảng cách chữ (tracking) âm là -4.8px và triệt tiêu hoàn toàn khoảng cách dòng (line-height: none) nhằm tạo ra sự sắc nét và ấn tượng mạnh mẽ.

Về mặt không gian bố cục, hệ thống duy trì các quy chuẩn giãn cách cực kỳ nghiêm ngặt. Lề ngang tổng thể của giao diện luôn được cố định ở mức 120px. Các khoảng cách giữa các khối thành phần cũng được chuẩn hóa, ví dụ như khoảng cách từ thanh điều hướng (Navbar) đến nội dung chính là 60px, và từ tiêu đề đến khung tìm kiếm là 44px.

2.5.4. Xử lý Hoạt ảnh Nâng cao với JavaScript Thuần

Một điểm khác biệt lớn về mặt kỹ thuật giao diện của dự án là việc xử lý các hoạt ảnh phức tạp (Animation), đặc biệt là lớp video nền tĩnh. Thay vì sử dụng các thuộc tính CSS Transition thông thường vốn dễ gây ra tình trạng giật lag, hệ thống bắt buộc sử dụng JavaScript thuần túy thông qua hàm `requestAnimationFrame`.

Luồng xử lý này kiểm soát các hiệu ứng xuất hiện (fade-in) kéo dài 250 mili-giây ngay khi video tải xong, và hiệu ứng biến mất (fade-out) được kích hoạt chính xác khi video còn 0.55 giây trước khi kết thúc.

Để đảm bảo tính liên tục của vòng lặp (loop), hệ thống sử dụng một biến trạng thái `fadingOutRef` nhằm ngăn chặn việc gọi lại hiệu ứng nhiều lần gây xung đột. Mỗi khi bắt đầu một hoạt ảnh mới, mọi khung hình trước đó đều bị hủy bỏ (cancel animation frames) để hiệu ứng chuyển cảnh tiếp nối chính xác từ độ mờ (opacity) hiện tại, mang lại một trải nghiệm thị giác vô cùng mượt mà mà không gặp tình trạng giật cục.

2.5.5. Cấu trúc Component và Tính năng Tương tác

Tuân theo nguyên tắc thiết kế của React, toàn bộ mã nguồn giao diện được phân tách thành các thành phần (Component) độc lập, rạch ròi về mặt logic và chức năng.

Kiến trúc giao diện bao gồm các lớp chính:

- Thành phần `VideoBackground` làm nền tảng hiển thị dưới cùng
- Thành phần `NavigationBar` quản lý thanh điều hướng
- Thành phần `HeroContent` tập trung hiển thị nội dung chính.

Bên trong `HeroContent`, thành phần `SearchInputComplex` được thiết kế như một không gian làm việc thu nhỏ với cấu trúc đa tầng, bao gồm phần hiển thị tín dụng giới hạn (credits), thanh nhập liệu ngôn ngữ tự nhiên và các công cụ tiện ích bổ sung như biểu tượng đánh kèm tệp hay tìm kiếm bằng giọng nói. Cách tổ chức này không chỉ giúp người dùng dễ dàng thao tác mà còn tối ưu hóa quá trình bảo trì và mở rộng tính năng của đội ngũ lập trình.

2.6. Công nghệ ảo hóa và Quản trị hạ tầng (DevOps)

Nhằm đảm bảo tính đồng bộ, sự ổn định và khả năng dễ dàng triển khai trên nhiều môi trường khác nhau, dự án Agent SQL không sử dụng phương pháp cài đặt hệ thống trực tiếp trên hệ điều hành vật lý. Thay vào đó, đề tài ứng dụng toàn diện công nghệ ảo hóa vùng chứa (Containerization) thông qua nền tảng Docker.

2.6.1. Tổng quan về Công nghệ Ảo hóa và Trình điều phối Docker Compose

Toàn bộ các phân hệ microservices của hệ thống, bao gồm API Gateway, dịch vụ xử lý AI, dịch vụ thực thi cơ sở dữ liệu (Query Service) và giao diện Frontend, đều được đóng gói thành các môi trường ảo hóa độc lập. Việc sử dụng vùng chứa giúp cô lập phần mềm cùng các thư viện phụ thuộc khỏi cấu hình hệ điều hành chủ, loại bỏ tình trạng mã nguồn chỉ chạy được trên một máy tính nhất định.

Cùng với đó, công cụ Docker Compose được sử dụng như một trình điều phối trung tâm. Thay vì phải khởi chạy từng vùng chứa một cách thủ công, hệ thống quản lý toàn bộ hệ sinh thái phần mềm thông qua tệp cấu hình duy nhất docker-compose.yml. Chỉ với một câu lệnh thực thi, toàn bộ các dịch vụ sẽ được đồng bộ hóa quá trình cài đặt, kết nối mạng và khởi động cùng lúc một cách mượt mà.

2.6.2. Quản lý Mạng nội bộ và Ánh xạ Cổng (Port Mapping)

Kiến trúc ảo hóa cho phép hệ thống thiết lập một ranh giới mạng nội bộ vô cùng chặt chẽ. Các dịch vụ bên trong dự án giao tiếp với nhau hoàn toàn qua giao thức HTTP nội bộ của hệ thống mạng Docker, từ đó ngăn chặn mọi rủi ro về việc gọi trực tiếp dữ liệu ra ngoài mạng Internet khi chưa được cấp phép.

Để người dùng và các công cụ bên ngoài có thể tương tác với hệ thống, kỹ thuật ánh xạ cổng (Port mapping) được áp dụng theo quy tắc kết nối cổng máy chủ vật lý với cổng vùng chứa. Cụ thể, hệ thống cố định Cổng giao tiếp trung tâm (API Gateway) tại số 8000 và giao diện người dùng tại cổng 3000. Các dịch vụ lõi như Query Service và NL2SQL được bảo vệ tại các cổng 8001 và 8002. Trong trường hợp máy chủ gặp sự cố do cổng mạng đã bị chiếm dụng, hệ thống cho phép kỹ sư thay đổi linh hoạt cổng ánh xạ bên ngoài mà không làm ảnh hưởng đến mã nguồn bên trong.

2.6.3. Cơ chế Nạp lại mã nguồn tức thời (Hot-Reload)

Nhằm tối ưu hóa hiệu suất làm việc của đội ngũ phát triển, môi trường ảo hóa được thiết lập đi kèm với cơ chế nạp lại mã nguồn tức thời (Hot-Reload) cho cả hệ thống Backend

lần Frontend. Thay vì phải biên dịch lại toàn bộ hình ảnh vùng chứa mỗi khi mã nguồn có sự thay đổi, hệ thống ứng dụng kỹ thuật gắn kết ổ đĩa ảo (Volume mounting).

Theo đó, thư mục chứa mã nguồn gốc trên máy tính của nhà phát triển được ánh xạ trực tiếp vào bên trong vùng chứa. Đối với Backend, máy chủ FastAPI tự động cập nhật những thay đổi nhờ vào tham số tải lại của Uvicorn. Đối với Frontend, hệ thống thiết lập một vùng chứa định danh (named volume) độc lập dành riêng cho thư mục chứa các thư viện tải về (node_modules). Kỹ thuật này giúp loại bỏ hoàn toàn các xung đột nhân hệ điều hành giữa máy chủ vật lý và vùng chứa, đảm bảo giao diện luôn được làm mới tức thì ngay khi nhà phát triển thao tác lưu mã lệnh.

2.6.4. Quản trị cấu hình và Biến môi trường bảo mật

Về mặt bảo mật hạ tầng, toàn bộ các cấu hình nhạy cảm như khóa kết nối lập trình (API Keys), thông tin đăng nhập hệ quản trị Supabase hay định tuyến các cổng mạng đều không bao giờ được viết hardcoded vào mã nguồn phần mềm. Thay vào đó, chúng được đưa vào quản lý tập trung thông qua các tập tin biến môi trường (.env).

Trình điều phối Docker Compose sẽ làm nhiệm vụ tự động nạp các biến số này vào từng vùng chứa tương ứng thông qua thuộc tính định danh môi trường (env_file). Phương pháp luận DevOps này giúp tách biệt hoàn toàn dữ liệu nhạy cảm khỏi kho lưu trữ mã nguồn, đảm bảo tính minh bạch và độ an toàn tuyệt đối khi hệ thống Agent SQL được luân chuyển qua lại giữa các môi trường kiểm thử và môi trường triển khai thực tế.

TÀI LIỆU THAM KHẢO

- [1] Google, “Google AI Studio – Gemini API Quickstart,” 2025. [Online]. Available: <https://ai.google.dev>
- [2] OpenAI, “OpenAI Platform Documentation,” 2025. [Online]. Available: <https://platform.openai.com/docs>
- [3] Groq, “Groq Cloud Documentation,” 2025. [Online]. Available: <https://console.groq.com/docs>
- [4] Docker Inc., “Docker Documentation,” 2025. [Online]. Available: <https://docs.docker.com>
- [5] Celery Project, “Celery: Distributed Task Queue Documentation,” 2025. [Online]. Available: <https://docs.celeryq.dev>
- [6] Redis Ltd., “Redis Documentation,” 2025. [Online]. Available: <https://redis.io/docs>
- [7] MinIO Inc., “MinIO: High Performance Object Storage Documentation,” 2025. [Online]. Available: <https://min.io/docs>
- [8] Cloudflare Inc., “Cloudflare R2 Documentation,” 2025. [Online]. Available: <https://developers.cloudflare.com/r2>
- [9] MongoDB Inc., “MongoDB Atlas Documentation; PyMongo & Motor Documentation,” 2025. [Online]. Available: <https://www.mongodb.com/docs>
- [10] Chroma, “Chroma Vector Database Documentation,” 2025. [Online]. Available: <https://docs.trychroma.com>
- [11] Vercel Inc., “Next.js 14 Documentation (App Router),” 2025. [Online]. Available: <https://nextjs.org/docs>
- [12] tiangolo, “FastAPI Framework Documentation,” 2025. [Online]. Available: <https://fastapi.tiangolo.com>